DOCUMENT RÉSERVÉ À L'ENSEIGNANT

Corrigé — Journaux de flux NetFlow/IPFIX

Réponses attendues, chronologie complète et notes de mise en œuvre

Confidentialité
Ne déposez ni ce corrigé ni `gen_scenario.py` dans le dépôt public des labos : le générateur décrit tout le scénario d'attaque. Seuls l'énoncé, `enquete.pcap.gz`, `enquete_flux.tar.gz` et `analyse_beacon.py` sont destinés aux étudiants.

Notes pour l'enseignant

Fichiers

Fichier	Rôle	Public
`enquete.pcap.gz`	Jeu d'enquête (3,9 Mo compressé, 173 Mo décompressé, 199 320 paquets complets)	Étudiants
`enquete_flux.tar.gz`	Les mêmes données déjà converties en flux avec nfpcapd 1.7.10 (dossier `enquete/`, 24 fichiers horaires, 10 681 flux)	Étudiants
`analyse_beacon.py`	Calcul de la régularité des connexions (intervalle moyen, coefficient de variation) ; compatible nfdump 1.7.3 et 1.7.10	Étudiants
`gen_scenario.py`	Générateur du pcap, reproductible (`--seed 42` par défaut)	Enseignant

Points techniques vérifiés

Tous les chiffres de ce corrigé ont été obtenus avec **nfdump 1.7.10**, la version de la Kali du Cyber Range. D'autres versions découpent les flux autrement : les volumes, les heures et les comportements restent identiques, mais les nombres d'enregistrements changent (10 681 flux en 1.7.10, mais 9 030 en 1.7.3, par exemple).

- **Paquets complets.** nfpcapd 1.7.10 rejette les paquets tronqués (« short caplen ») : une première version du jeu, aux paquets tronqués, perdait ainsi 97 % des octets et toute l'exfiltration. Le pcap actuel contient des paquets complets, dont la charge utile est remplie de zéros (d'où la bonne compression).
- **Le dernier ACK forme un enregistrement à part.** Avec nfpcapd 1.7.10, après la fermeture d'une connexion TCP (FIN), l'ACK final arrive dans un nouvel enregistrement d'un paquet (40 octets). Une tentative de force brute donne donc deux lignes. L'option `-a` les recolle.
- **Convertir le pcap avec nfpcapd**, pas avec `softflowd -r` : en lecture de fichier, softflowd 1.1.0 produit des horodatages faux.
- **Fuseau horaire.** Le scénario est en heure de Gatineau (HAE). Sur une Kali réglée en UTC, toutes les heures sont décalées de 4 h sans `export TZ=America/Toronto`.

- **Délais de nfpcapd** : 300 s (actif) et 60 s (inactif). Les sessions longues (SSH, RDP, exfiltration) sont découpées en tranches d'environ 5 min.
- **Filtres net** : le réseau doit être aligné sur son masque. `10.0.0.100/27` ne correspond à rien en 1.7.10 ; il faut `10.0.0.96/27` (de .96 à .127, ce qui couvre les postes .101 à .110).
- **L'option -t est héritée en 1.7.10.** Elle ne garde que les flux entièrement compris dans la fenêtre. Le filtre `'first seen >= 2026-10-14T17:30'` est plus sûr ; il n'existe pas en 1.7.3.
- **Ports éphémères uniques.** Le générateur ne réutilise jamais un port source dans la journée, et le client NTP utilise un port éphémère (comme chrony) : sinon, nfpcapd 1.7.10 fusionnait des connexions distinctes en flux de plusieurs heures.
- **Adresses et domaines.** Toutes les adresses externes sont des plages de documentation (RFC 5737) et les domaines suspects utilisent le TLD réservé `.example`.

Variantes

`python3 gen_scenario.py --seed 7 --out enquete.pcap` génère un autre trafic de fond. Les heures clés de l'attaque restent les mêmes, mais certains décomptes (connexions du beacon, requêtes du tunnel, flux de fond) varient légèrement. Refaites alors les vérifications avec les commandes de ce corrigé, et régénérez l'archive de flux avec `nfpcapd -r enquete.pcap -w enquete -t 3600`.

Partie 1 sur le pfSense

Le menu **Diagnostics > Command Prompt** de pfSense permet aux étudiants d'exécuter `softflowctl … expire-all` sans passer par la console. Le nom du socket dans `/var/run/` devrait être identique dans tous les environnements, puisque les pfSense sont clonés.

Partie 1 — Réponses attendues

Les sorties ci-dessous ont été obtenues dans le Cyber Range le 27 octobre 2026 (Kali 10.0.0.2). Les ports, les adresses d'example.com et les volumes varient d'un essai à l'autre ; le nombre d'enregistrements et de paquets DNS reste le même.

Q1a. **Deux enregistrements**, un par sens de la connexion TCP. Un flux est unidirectionnel.

```
nfdump -R ~/flux_curl -o long -O tstart 'port 53154'

                                                    paquets  octets
TCP  10.0.0.2:53154      -> 172.66.147.243:443  ...AP.SF  15    2842
TCP  172.66.147.243:443  -> 10.0.0.2:53154      ...AP.SF  13    6531
```

Q1b. Deux flux **UDP entre la Kali et le pfSense (10.0.0.1:53)** : la requête DNS et sa réponse. Un filtre sur l'adresse d'example.com ne les montre pas, puisque la résolution se fait avec le résolveur et non avec le site. Il faut filtrer sur le client ou sur le port DNS : `'host 10.0.0.2'`, `'port 53'` ou `'proto udp'`.

```
nfdump -R ~/flux_curl -o long -O tstart 'port 53'

                                                    paquets  octets
UDP  10.0.0.2:53536      -> 10.0.0.1:53         ........  2      114
UDP  10.0.0.1:53         -> 10.0.0.2:53536      ........  2      202
```

Les quatre flux portent la même heure de début (16:51:48 dans cet essai) : c'est ce qui permet de relier la résolution DNS à la connexion.

Q1c. **Quatre enregistrements au total.** La Kali demande l'adresse IPv4 (requête A) et l'adresse IPv6 (requête AAAA) d'example.com, deux requêtes envoyées par le **même port source**. Elles ont donc le même 5-tuple et sont comptées dans le même flux : 2 paquets par sens (114 octets = 2 × 57). Une capture tcpdump sur la Kali le confirme : `A? example.com` et `AAAA? example.com` partent toutes deux du même port.

- **Leçon visée** : une seule action (« visiter un site ») produit des conversations avec plusieurs interlocuteurs. Pour la reconstituer, on part du client et de l'heure, pas seulement du serveur.
- **Un autre nombre** veut dire que le trafic n'a pas été isolé : Firefox ouvert, `dig` lancé en même temps, ou flux d'une autre commande encore dans le dossier.
- **Aucun flux DNS** : si la Kali avait déjà le nom en cache (par exemple avec systemd-resolved), aucune requête ne sort sur le réseau. Un flux n'existe que si un paquet traverse le point d'observation.

Q2. Pour une requête `-I`, les volumes sont faibles, mais c'est normalement le **serveur qui envoie le plus** (certificat TLS et réponse). C'est le profil d'une navigation ou d'un téléchargement normal.

Q3. softflowd garde les flux dans son cache jusqu'à leur expiration : fin de connexion (FIN ou RST, puis délai), délai d'inactivité ou délai actif. Ses délais par défaut sont longs, de l'ordre de quelques minutes. `expire-all` force l'export immédiat. S'ajoute ensuite la rotation de nfcapd toutes les 5 minutes.

Q4. C'est l'**adresse de la Kali** (10.0.0.2), car softflowd observe l'interface LAN, avant la traduction d'adresses. Sur l'interface WAN, la source serait l'adresse WAN du pfSense (10.30.x.x dans le range) pour tous les postes : on perdrait l'identité de la machine interne.

Q5. Dix connexions vers l'adresse d'example.com, espacées d'environ 15 s plus la durée de chaque curl, avec des volumes presque identiques. C'est la signature d'un beacon : régularité de l'intervalle et de la taille, vers une même destination.

Partie 2 — Réponses attendues

Q6. Vue d'ensemble.

```
nfdump -R ~/enquete -s proto/flows
nfdump -R ~/enquete -s srcip/flows -n 5
```

- **10 681 flux**, du 14 octobre 2026 à 00:03:17 jusqu'à 23:58:36 (TCP : 7 112, UDP : 3 304, ICMP : 265), pour 167 Mo au total.
- Top 5 des sources : **10.0.0.105** (4 352 flux, 40,7 %), 10.0.0.11 (1 187), 198.51.100.77 (912), 192.0.2.53 (900), 192.0.2.123 (403).
- Le poste **10.0.0.105** génère à lui seul 40 % de tous les flux du réseau : c'est anormal pour un poste parmi dix.
- Le serveur **10.0.0.11** arrive deuxième, mais surtout parce qu'il **répond** au balayage de ports. La démonstration suit.

Pourquoi 10.0.0.11 a-t-il autant de flux ?

Le classement par source donne un total, pas son origine. On décompose les flux émis par la machine, d'abord par destination, puis par port :

```
nfdump -R ~/enquete 'src ip 10.0.0.11' -s dstip/flows -n 5
nfdump -R ~/enquete 'src ip 10.0.0.11' -s dstport/flows -n 3
```

Destination	Flux	Part	Ce que c'est
10.0.0.105, port 58417	1 024	86 %	Réponses au balayage SYN (de 09:05:02 à 09:05:09)
10.0.0.105, autres ports	64	5 %	Réponses aux sessions SSH de l'attaquant (63) et au ping (1)
192.0.2.123:123	85	7 %	NTP du serveur
10.0.0.110	6	< 1 %	SSH de l'administrateur
203.0.113.200 et 203.0.113.50	7	< 1 %	Exfiltration et mise à jour
10.0.0.1:53	1	< 1 %	Requête DNS avant la mise à jour
Total	**1 187**		

La confirmation vient des drapeaux :

```
nfdump -R ~/enquete -o long -O tstart 'src ip 10.0.0.11 and dst port 58417'

10.0.0.11:174 -> 10.0.0.105:58417   ...A.R..   1 paquet   40 octets
10.0.0.11:271 -> 10.0.0.105:58417   ...A.R..   1 paquet   40 octets
```

- `...A.R..` = **ACK + RST** : la réponse d'un serveur à un SYN reçu sur un **port fermé**. 1 023 des 1 024 flux sont des RST ; le dernier est le SYN-ACK du port 22, le seul ouvert.

- Chaque flux sort d'un **port de service différent** (174, 271, 954…) vers **un seul port** du poste : quelqu'un a frappé à toutes les portes, et le serveur a répondu à chaque fois.
- Ce sont les 1 024 sondes de Q8, vues du côté de la victime. Sans le balayage, 10.0.0.11 n'aurait que 163 flux.
- **Leçon visée** : le nombre de flux mesure l'activité, pas la culpabilité. 10.0.0.11 est ici une victime ; ce sont ses **octets** qui le trahiront à Q13.

Q7. Ligne de base.

Une ligne de base décrit le trafic **normal**. Il faut donc d'abord retirer ce qu'on soupçonne déjà : le poste 10.0.0.105, repéré à Q6 (`not host 10.0.0.105`), et la soirée, pour les volumes (`first seen` entre 8 h et 17 h 30). Sans ces précautions, l'exfiltration du soir se retrouve dans la « normale » du serveur. On répond ensuite à quatre questions, une commande chacune.

1. Quels services internes sont utilisés, et par qui ?

```
nfdump -R ~/enquete -A srcip,dstip,dstport -O flows -n 30 \
  'src net 10.0.0.96/27 and dst net 10.0.0.0/28 and not host 10.0.0.105'
```

`-A srcip,dstip,dstport` donne une ligne par couple « poste → serveur:port » ; le filtre garde les postes (10.0.0.96/27) qui parlent aux serveurs internes (10.0.0.0/28, soit .1 à .15). Résultat :

- **DNS** : les 9 postes interrogent **10.0.0.1:53** (30 à 38 requêtes chacun, 64 octets en moyenne).
- **SMB** : les 9 postes se connectent à **10.0.0.10:445**, et à aucun autre poste (8 à 16 flux chacun).
- **SSH** : seul **10.0.0.110** (l'administrateur) se connecte à **10.0.0.11:22** (6 flux, dès 08:31).

2. Quels services Internet sont utilisés ?

```
nfdump -R ~/enquete -s dstport:p/flows -n 5 \
  'src net 10.0.0.0/24 and not dst net 10.0.0.0/24 and not host 10.0.0.105'
```

Trois services seulement : **HTTPS** (TCP 443, 75 % des flux), **NTP** (UDP 123) et **HTTP** (TCP 80, deux flux : la mise à jour du serveur Linux à 12:05). Aucun DNS vers l'extérieur.

3. Vers quelles destinations ?

```
nfdump -R ~/enquete -s dstip/flows -n 20 \
  'src net 10.0.0.0/24 and not dst net 10.0.0.0/24 and not host 10.0.0.105'
```

Une quinzaine de sites Web visités par tous les postes, le serveur NTP **192.0.2.123** et le miroir de mises à jour **203.0.113.50**. Une destination détonne déjà : **203.0.113.200**, avec seulement 5 flux mais 93 % des octets. Elle sera expliquée à Q13.

4. Quels volumes, et à quelles heures ?

Le filtre devient long : on le range dans une variable, puis on l'utilise entre guillemets doubles.

```
JOUR='first seen >= 2026-10-14T08:00 and first seen < 2026-10-14T17:30 and not host
10.0.0.105'
nfdump -R ~/enquete -s srcip/bytes -n 12 "src net 10.0.0.0/24 and not dst net 10.0.0.0/24
and $JOUR"
nfdump -R ~/enquete -s dstip/bytes -n 12 "dst net 10.0.0.0/24 and not src net 10.0.0.0/24
and $JOUR"
```

Les colonnes **Date first seen** et **Duration** d'un `-s srcip/flows` donnent la période d'activité de chaque poste.

La ligne de base obtenue

Élément	Normal pour ce réseau
Heures d'activité des postes	Premiers flux de 08:03 à 08:22, derniers vers 17 h ; rien la nuit
DNS	Uniquement vers 10.0.0.1, environ 35 requêtes par poste et par jour, 64 octets chacune
Partages de fichiers	Postes → 10.0.0.10:445 seulement ; jamais de poste à poste
Administration	SSH vers 10.0.0.11 depuis 10.0.0.110 seulement
Internet	HTTPS vers une quinzaine de sites, NTP vers 192.0.2.123 ; HTTP seulement pour la mise à jour du serveur
Volumes d'un poste (8 h - 17 h 30)	Environ 250 à 420 Ko envoyés et 5 à 11 Mo reçus vers ou depuis Internet : un poste reçoit beaucoup plus qu'il n'envoie
Volumes du serveur 10.0.0.11 (8 h - 17 h 30)	Environ 45 Ko envoyés (NTP, requêtes) et 2,5 Mo reçus (mise à jour)

Cette grille servira de référence pour Q8 à Q13 : chaque comportement de l'attaque s'en écarte sur au moins un point (heure, destination, service, sens du volume).

Q8. Reconnaissance (MITRE ATT&CK T1018 et T1046).

```
nfdump -R ~/enquete 'proto icmp' -s srcip/flows -n 3
nfdump -R ~/enquete 'flags S and not flags A' -s srcip/flows
nfdump -R ~/enquete 'dst ip 10.0.0.105 and dst port 58417 and flags S' -o line -O tstart
```

- **09:03:41** : balayage ICMP de **253 adresses** du /24 en 2,8 s depuis 10.0.0.105 ; 12 hôtes répondent.
- **09:05:02 à 09:05:09** : balayage SYN des **ports 1 à 1 024** de 10.0.0.11, depuis le port source fixe 58417 (1 paquet de 44 octets par flux, typique de `nmap -sS`). Seul le port **22** est ouvert.
- **09:07:30** : balayage de **20 ports** de 10.0.0.10 ; ports ouverts : **135, 139, 445, 3389, 5985**.
- Au total, 1 044 flux SYN sans ACK, tous depuis 10.0.0.105.

Q9. Accès initial par force brute SSH (T1110).

```
nfdump -R ~/enquete -a 'src ip 10.0.0.105 and dst ip 10.0.0.11 and dst port 22' \
       -t '2026/10/14.09:20:00-2026/10/14.09:45:00' -o line -O tstart
```

- **57 tentatives** de 09:21:10 à 09:22:28, environ une toutes les **1,4 s**.

- Chaque tentative : environ 1,2 s, 7 paquets et **2,57 Ko** du client vers le serveur (environ 3,48 Ko au retour). La taille quasi identique est la signature recherchée.
- Sans `-a`, chaque tentative occupe deux lignes : la connexion (6 paquets) et l'ACK final (1 paquet de 40 octets), exporté à part. Les deux lectures sont acceptables si l'étudiant l'explique.
- **09:22:30** : la force brute s'arrête et une session de **18 min 30 s** commence (jusqu'à 09:41 environ), avec environ 53 Ko envoyés et 392 Ko reçus. C'est une connexion réussie très probable : l'attaquant a trouvé le mot de passe et travaille de façon interactive.
- Sans `-a`, cette session apparaît en plusieurs enregistrements par sens (tranches d'environ 5 min, délai actif de 300 s).

Q10. Mouvement latéral (T1021.002 et T1021.001).

```
nfdump -R ~/enquete -a 'src net 10.0.0.96/27 and dst net 10.0.0.96/27 and \
       (dst port 445 or dst port 3389 or dst port 5985)' -o line -O tstart
```

- **09:52:18 à 09:54:00** : connexions SMB (445) de 10.0.0.105 vers quatre postes, **10.0.0.102, .103, .104 et .107**, avec environ 215 à 235 Ko envoyés à chacun (dépôt possible d'outils).
- **09:58:44** : session RDP (3389) vers **10.0.0.108** pendant 10 min 40 s (385 Ko envoyés, 4,9 Mo reçus).
- Selon la ligne de base, les postes ne parlent SMB qu'au serveur de fichiers : toute connexion d'un poste à un autre poste est suspecte.
- Attention au filtre : `10.0.0.100/27` ne donne aucun résultat, car le réseau doit commencer sur une frontière du masque. `10.0.0.96/27` couvre les postes .101 à .110. Un étudiant peut aussi lister les postes un par un.

Q11. Commande et contrôle (T1071.001).

```
python3 analyse_beacon.py ~/enquete
```

Source	Destination	Port	Conn.	Interv. moy.	CV interv.	Octets moy.
10.0.0.101 à .110	192.0.2.123	123/UDP	31 ou 32 chacun	1 024,0 s	0,000	76
10.0.0.105	198.51.100.77	443/TCP	912	60,0 s	0,060	721
10.0.0.105	192.0.2.53	53/UDP	900	2,1 s	0,138	176

- Destination : **198.51.100.77:443**, contactée **912 fois** de **08:47:12 à 23:58:35** (plus de 15 h, bien après les heures de travail).
- Intervalle moyen de **60,0 s** avec une gigue d'environ ±10 % (CV de 0,060), et environ **721 octets** par connexion (CV de 0,008).
- Une requête DNS de 10.0.0.105 vers 10.0.0.1 a lieu à **08:47:10**, 2 s avant la première connexion : c'est la résolution du domaine du serveur de C2.

- **NTP en tête** : son intervalle est parfaitement régulier (1 024 s, CV de 0), mais la destination est le serveur NTP officiel, sur le port 123, depuis tous les postes. C'est un faux positif classique ; une liste blanche est nécessaire.
- Le script recolle les morceaux d'une même connexion (ACK final, tranches de 5 min) : il compte des **connexions**, pas des enregistrements. C'est pourquoi `nfdump 'dst ip 198.51.100.77'` affiche environ deux fois plus de lignes.

Q12. Tunnel DNS (T1071.004).

```
nfdump -R ~/enquete 'proto udp and dst port 53 and not dst ip 10.0.0.1' -s srcip/flows
nfdump -R ~/enquete 'proto udp and dst ip 10.0.0.1 and dst port 53' -s proto/flows
```

- **900 requêtes** de 10.0.0.105 vers **192.0.2.53:53**, un résolveur externe non autorisé, de **10:20:05 à 10:51:40**, environ toutes les 2,1 s.
- Taille moyenne de **176 octets** par requête, contre **64 octets** pour les requêtes normales vers 10.0.0.1. Les réponses font en moyenne 263 octets.
- Le résolveur interne est le seul autorisé : un poste qui interroge directement un DNS externe, à ce rythme et avec des requêtes presque trois fois plus grosses, transporte vraisemblablement des données dans les noms de domaine.

Q13. Exfiltration (T1048.002).

```
nfdump -R ~/enquete -t '2026/10/14.17:30:00-2026/10/14.23:59:59' \
       'src net 10.0.0.0/24 and not dst net 10.0.0.0/24' -s srcip/bytes
nfdump -R ~/enquete -a 'host 203.0.113.200' -o line -O tstart
nfdump -R ~/enquete -a 'src ip 10.0.0.105 and dst ip 10.0.0.11 and dst port 22' \
       -t '2026/10/14.22:00:00-2026/10/14.23:00:00' -o line -O tstart
```

- **10.0.0.11** (SRV-LINUX) envoie environ **43,1 Mo** vers **203.0.113.200:443** de **22:14:07 à 22:31:00**. Sans `-a`, ce sens occupe 5 enregistrements (4 tranches et l'ACK final). Il ne reçoit qu'environ 1,3 Mo en retour : des accusés de réception.
- Pour ce serveur, la normale est l'inverse : il reçoit des mises à jour et n'envoie presque rien vers Internet (NTP et environ 43 Ko vers le miroir).
- Accès : à **22:09:51**, une session SSH part de **10.0.0.105** vers 10.0.0.11 et dure environ 26 min. Le poste compromis, resté allumé, est toujours relié au C2 : l'attaquant s'en sert comme rebond pour lancer l'exfiltration depuis le serveur.
- Le filtre `'first seen >= 2026-10-14T17:30'` donne le même classement que `-t` avec nfdump 1.7.10.

Bonus. Dans Wireshark, le filtre `dns.qry.name contains "cdn-telemetrie"` montre la requête de **08:47:10** pour **api.cdn-telemetrie.example**, résolue en 198.51.100.77, et les requêtes TXT du tunnel vers des sous-domaines aléatoires de **t.cdn-telemetrie.example**. Les flux ont permis de cibler l'heure et les hôtes ; le pcap confirme ce qui a été transmis.

Partie 3 — Éléments de correction

A. Chronologie de référence

Heure	Source → destination	Comportement	ATT&CK
08:47:10	10.0.0.105 → 10.0.0.1:53	Résolution du domaine du C2	—
08:47:12	10.0.0.105 → 198.51.100.77:443	Début du beaconing (toutes les ~60 s, jusqu'à 23:58)	T1071.001
09:03:41	10.0.0.105 → 10.0.0.0/24	Balayage ICMP de 253 adresses	T1018
09:05:02	10.0.0.105 → 10.0.0.11	Balayage SYN de 1 024 ports (22 ouvert)	T1046
09:07:30	10.0.0.105 → 10.0.0.10	Balayage de 20 ports Windows	T1046
09:21:10	10.0.0.105 → 10.0.0.11:22	Force brute SSH, 57 tentatives	T1110
09:22:30	10.0.0.105 → 10.0.0.11:22	Session SSH réussie (~18 min)	T1078
09:52:18	10.0.0.105 → .102, .103, .104, .107:445	Connexions SMB entre postes	T1021.002
09:58:44	10.0.0.105 → 10.0.0.108:3389	Session RDP (~11 min)	T1021.001
10:20:05	10.0.0.105 → 192.0.2.53:53	Tunnel DNS, 900 requêtes en 31 min	T1071.004
22:09:51	10.0.0.105 → 10.0.0.11:22	Retour de l'attaquant en SSH (hors heures)	T1021.004
22:14:07	10.0.0.11 → 203.0.113.200:443	Exfiltration d'environ 43 Mo (jusqu'à 22:31)	T1048.002

B. Exemples de règles de détection

Règle	Filtre ou méthode	Seuil	Faux positifs probables
Balayage de ports	`'flags S and not flags A'` avec `-s srcip/flows`	Plus de 100 flux par source en 5 min	Scanner de vulnérabilités autorisé, outil d'inventaire (liste blanche)
DNS non autorisé	`'proto udp and dst port 53 and not dst ip 10.0.0.1'`	Tout flux	Appareil ou application avec un DNS codé en dur, client VPN
Beaconing	`analyse_beacon.py`	CV inférieur à 0,1 et au moins 50 connexions vers une destination hors liste blanche	NTP, mises à jour, télémétrie légitime
Mouvement latéral	`'src net 10.0.0.96/27 and dst net 10.0.0.96/27 and (dst port 445 or dst port 3389 or dst port 5985)'`	Tout flux, sauf depuis le poste de l'administrateur	Soutien technique à distance, partages ponctuels entre postes

Règle	Filtre ou méthode	Seuil	Faux positifs probables
Volume sortant inhabituel	`'src net 10.0.0.0/24 and not dst net 10.0.0.0/24'` avec `-s srcip/bytes` et `-t` hors heures	Plus de 10 Mo par hôte hors heures, ou tout envoi notable d'un serveur qui n'envoie normalement rien	Sauvegardes infonuagiques, synchronisation OneDrive

C. Recommandations attendues

- **Confinement** : isoler 10.0.0.105 et 10.0.0.11, bloquer 198.51.100.77 et 203.0.113.200, et chercher ces adresses dans l'historique des flux d'autres périodes.
- **Comptes** : réinitialiser les mots de passe et les clés SSH du serveur ; exiger l'authentification par clé et limiter les tentatives (fail2ban, par exemple).
- **Durcissement réseau** : forcer le DNS par le résolveur interne (bloquer le port 53 sortant ailleurs) ; bloquer SMB, RDP et WinRM entre postes ; restreindre l'accès SSH au serveur au seul poste de l'administrateur.
- **Visibilité** : conserver les flux plus longtemps et y attacher des alertes automatiques à partir des règles ci-dessus ; enrichir avec les journaux DNS du pfSense.
- **Suite de l'enquête** : une analyse forensique de 10.0.0.11 est nécessaire pour savoir quelles données ont été volées. Les flux donnent le volume et la destination, pas le contenu.